JPQL examples – 7 aprile 2025

TBD: some comments to be translated to English

Sommario

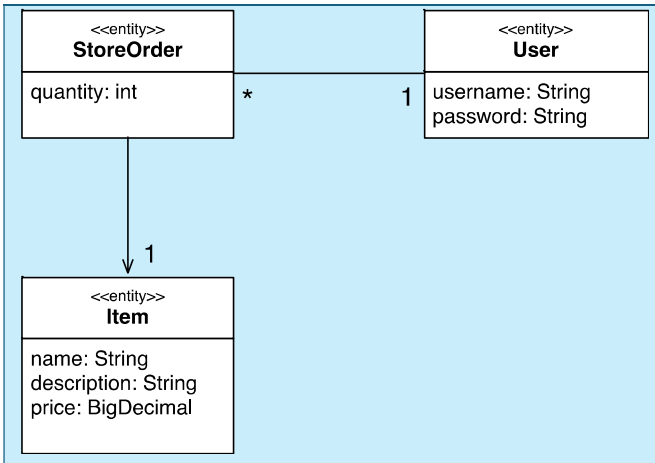
1 Refined version – derived from the first round 1

2 First round – to be refined 15

5 Further experiments 23

1 Refined version – derived from the first round

Ripete il first round forzando di avere una corretta implementazione dello schema UML (per le molteplicità) ... e prevenendo altre digressioni osservate



can you write the java classes that implement the following schema, with JPA annotations that map them on the database? In so doing, please consider that StoreOrder has an association @ManyToOne to the User, and viceversa the User has an association @OneToMany to StoreOrder, and this bidirectional association shall be annotated using MappedBy

```
import javax.persistence.*;
import java.math.BigDecimal;
import java.util.List;

@Entity
public class StoreOrder {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    private int quantity;

    @ManyToOne
    @JoinColumn(name = "user_id")
    private User user;

    // Getters and Setters
    …
}

@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String username;
    private String password;

    @OneToMany(mappedBy = "user")
    private List<StoreOrder> storeOrders;

    // Getters and Setters
    …
}

@Entity
public class Item {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String description;
    private BigDecimal price;

    // Getters and Setters
    …
}
```

StoreOrder shall also be associated to Item, which should be a @ManyToOne (unidirectional) association. Can you correct?

```
import javax.persistence.*;

@Entity
public class StoreOrder {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private int quantity;

    @ManyToOne
    @JoinColumn(name = "user_id")
    private User user;

    @ManyToOne
    @JoinColumn(name = "item_id")
    private Item item;

    // Getters and Setters
    …
}
```

can you write the Data Access Objects associated with these classes, using JPA? In so doing, please do not use the Spring Framework. In the UserDAO please include a method that retrieves all the User that have at least a StoreOrder that is associated with an Item passed as a parameter :item

```
import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;
import java.util.List;

public class UserDAO {
    private EntityManagerFactory emf =
Persistence.createEntityManagerFactory("example-unit");

    public void save(User user) {
        EntityManager em = emf.createEntityManager();
        em.getTransaction().begin();
        em.persist(user);
        em.getTransaction().commit();
        em.close();
    }

    public User findById(Long id) {
        EntityManager em = emf.createEntityManager();
        User user = em.find(User.class, id);
        em.close();
        return user;
    }

    public List<User> findAll() {
        EntityManager em = emf.createEntityManager();
        List<User> users = em.createQuery("SELECT u FROM User u",
User.class).getResultList();
        em.close();
        return users;
    }

    public List<User> findUsersByItem(Item item) {
        EntityManager em = emf.createEntityManager();
        List<User> users = em.createQuery(
            "SELECT DISTINCT u FROM User u " +
            "JOIN u.storeOrders s " +
            "WHERE s.item = :item", User.class)
            .setParameter("item", item)
            .getResultList();
        em.close();
        return users;
    }
```

FixMe: si poteva scrivere anche senza usare JOIN come "SELECT s.user FROM StoreOrder s WHERE s.item = :item"? (la differenza è il FROM da cui parte)

```
    public void delete(User user) {
        EntityManager em = emf.createEntityManager();
        em.getTransaction().begin();
        user = em.merge(user);
        em.remove(user);
        em.getTransaction().commit();
        em.close();
    }
}
```

NB: nota l'uso di merge() e remove() … richiede che user sia già stato nel PersistenceContext, poi funziona anche se ora è detached … ma ci vuole comunque che un find o una query abbiano associato l'oggetto user a una database identity

```
import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;
import java.util.List;

public class StoreOrderDAO {
    private EntityManagerFactory emf =
Persistence.createEntityManagerFactory("example-unit");

    public void save(StoreOrder storeOrder) {
        EntityManager em = emf.createEntityManager();
        em.getTransaction().begin();
        em.persist(storeOrder);
        em.getTransaction().commit();
        em.close();
    }

    public StoreOrder findById(Long id) {
        EntityManager em = emf.createEntityManager();
        StoreOrder storeOrder = em.find(StoreOrder.class, id);
        em.close();
        return storeOrder;
    }

    public List<StoreOrder> findAll() {
        EntityManager em = emf.createEntityManager();
        List<StoreOrder> storeOrders = em.createQuery("SELECT s FROM
StoreOrder s", StoreOrder.class).getResultList();
        em.close();
        return storeOrders;
    }

    public void delete(StoreOrder storeOrder) {
```

```
        EntityManager em = emf.createEntityManager();
        em.getTransaction().begin();
        storeOrder = em.merge(storeOrder);
        em.remove(storeOrder);
        em.getTransaction().commit();
        em.close();
    }
}
```

```
import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;
import java.util.List;

public class ItemDAO {
    private EntityManagerFactory emf =
Persistence.createEntityManagerFactory("example-unit");

    public void save(Item item) {
        EntityManager em = emf.createEntityManager();
        em.getTransaction().begin();
        em.persist(item);
        em.getTransaction().commit();
        em.close();
    }

    public Item findById(Long id) {
        EntityManager em = emf.createEntityManager();
        Item item = em.find(Item.class, id);
        em.close();
        return item;
    }

    public List<Item> findAll() {
        EntityManager em = emf.createEntityManager();
        List<Item> items = em.createQuery("SELECT i FROM Item i",
Item.class).getResultList();
        em.close();
        return items;
    }

    public void delete(Item item) {
        EntityManager em = emf.createEntityManager();
        em.getTransaction().begin();
        item = em.merge(item);
        em.remove(item);
        em.getTransaction().commit();
        em.close();
    }
}
```

FixMe: Avendo chiesto di non usare Spring, lui ha inteso che non puo' usare dependency injection per avere un managed EntityManager ... e quindi crea una EntityManagerFactory ogni volta che crea un DAO ... che è molto inefficiente

Si potrebbe avere injection, usando "CDI in purezza" con un Producer di EntityManagerFactory, come descritto ad esempio su **stackoverflow**

https://stackoverflow.com/questions/40469121/can-i-inject-an-jpa-entitymanager-using-cdi-and-persistencecontext-like-with-s

```
You can do it this way: create Entity Manager Factory Producer

public class EntityManagerFactoryProducer {

    @Produces
    @ApplicationScoped
    public EntityManagerFactory create() {
        return Persistence.createEntityManagerFactory("PU");
    }

    public void destroy(@Disposes EntityManagerFactory factory) {
        factory.close();
    }

}

and create Entity Manager Producer

public class EntityManagerProducer {

    @Inject
    transient EntityManagerFactory emf;

    @Produces
    @RequestScoped
    public EntityManager create() {
        return emf.createEntityManager();
    }

    public void destroy(@Disposes EntityManager em) {
        em.close();
    }
}

then you can use dependency injection (CDI will create one em for each
request)

@Inject
EntityManager entityManager;

ps. If you have more then one PU use @Qualifier
```

Non avendo ancora introdotto CDI (... e i suoi Producers), conviene perora usare @PersistenceContext di Spring

Can you rewrite these DAOs using @PersistenceContext to receive the EntityManager by Injection? In so doing, can you also add a BaseDAO that maintains the EntityManager and let all the DAOs extend it?

```
import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;

public abstract class BaseDAO {
    @PersistenceContext
    protected EntityManager entityManager;

NB: un caso interessante di uso di protected. Serve se i DAOs sono tenuti
in packages distinti

    // You can include shared utility methods here, if needed
NB: sì, converrebbe farlo, usando i generics, per tutti i metodi che
hanno implementazione omogenea, e.g. findById(), findAll(), remove() …
Cmq, già quita si nota quanto vengono alleggeriti i DAO.
}
```

```
import javax.transaction.Transactional;
import java.util.List;

NB: avendo accettato di usare @PersistenceContext è venuto con lui anche
@Transactional … non era necessario

@Transactional
public class UserDAO extends BaseDAO {

    public void save(User user) {
        entityManager.persist(user);
    }

    public User findById(Long id) {
        return entityManager.find(User.class, id);
    }

    public List<User> findAll() {
        return entityManager.createQuery("SELECT u FROM User u",
User.class).getResultList();
    }

    public List<User> findUsersByItem(Item item) {
        return entityManager.createQuery(
            "SELECT DISTINCT u FROM User u " +
            "JOIN u.storeOrders s " +
            "WHERE s.item = :item", User.class)
```

```
            .setParameter("item", item)
            .getResultList();
    }

    public void delete(User user) {
        user = entityManager.merge(user);
        entityManager.remove(user);
    }
}
```

```
import javax.transaction.Transactional;
import java.util.List;

@Transactional
public class StoreOrderDAO extends BaseDAO {

    public void save(StoreOrder storeOrder) {
        entityManager.persist(storeOrder);
    }

    public StoreOrder findById(Long id) {
        return entityManager.find(StoreOrder.class, id);
    }

    public List<StoreOrder> findAll() {
        return entityManager.createQuery("SELECT s FROM StoreOrder s",
StoreOrder.class).getResultList();
    }

    public void delete(StoreOrder storeOrder) {
        storeOrder = entityManager.merge(storeOrder);
        entityManager.remove(storeOrder);
    }
}
```

```
import javax.transaction.Transactional;
import java.util.List;

@Transactional
public class ItemDAO extends BaseDAO {

    public void save(Item item) {
        entityManager.persist(item);
    }

    public Item findById(Long id) {
        return entityManager.find(Item.class, id);
    }

    public List<Item> findAll() {
```

```
        return entityManager.createQuery("SELECT i FROM Item i",
Item.class).getResultList();
    }

    public void delete(Item item) {
        item = entityManager.merge(item);
        entityManager.remove(item);
    }
}
```

Explanation of Changes:

BaseDAO:

- Centralizes the EntityManager management using @PersistenceContext injection.
- Encourages reusability and consistency across DAOs.

@PersistenceContext:

- Injects the EntityManager directly into the BaseDAO.
- Removes the need for manually instantiating EntityManagerFactory and EntityManager.

Transactional Management:

- Annotated with @Transactional to ensure database operations are executed within a transaction scope.
- This ensures operations such as persist, merge, and remove are handled atomically.

Could the query
"SELECT DISTINCT u FROM User u " + "JOIN u.storeOrders s " +"WHERE s.item = :item"
be written as:
“SELECT s.user FROM StoreOrder s WHERE s.item = :item”
?

Yes, you're correct! The query:

```
SELECT DISTINCT u FROM User u JOIN u.storeOrders s WHERE s.item = :item
```

can be simplified to:

```
SELECT DISTINCT s.user FROM StoreOrder s WHERE s.item = :item
```

Both achieve the same result of fetching users who have at least one StoreOrder associated with the given Item.

Explanation:

- In the simplified version, you directly query the StoreOrder table and retrieve its associated User through the s.user field.

- The DISTINCT keyword ensures that duplicate users are excluded from the result set, as there might be multiple StoreOrder entries pointing to the same User.

This approach is more concise and avoids the need to explicitly join the User and StoreOrder entities, as the association between User and StoreOrder is already captured in the mapping.

would this simplification impact also on queries that well be performed on the database, and thus on performance?

Yes, the simplification could have implications for database query execution and performance, depending on the context and database design. Here's an analysis of the potential impact:

1. Query Optimization

- **The simplified query directly accesses the StoreOrder table** (SELECT DISTINCT s.user FROM StoreOrder s WHERE s.item = :item) instead of performing a join across User and StoreOrder. This reduces the complexity of the query.
- For databases optimized for **indexed foreign keys,** querying directly from the StoreOrder table may result in faster execution since the query avoids the additional step of joining tables.

2. Database Indexing

- If the foreign key s.item in the StoreOrder table is indexed, the database can efficiently locate all matching rows for the given Item. The retrieval of distinct User objects (s.user) would then leverage the foreign key reference to minimize overhead.

3. Result Set Handling

- **The original query** (SELECT DISTINCT u FROM User u JOIN u.storeOrders s WHERE s.item = :item) involves **fetching User objects explicitly via a join**, which might load additional user-related fields unnecessarily if you're only interested in the distinct users.
- **The simplified query may reduce data transfer** overhead since it focuses only on StoreOrder's user field, avoiding fetching unrelated fields unless specified by JPA or Hibernate.

4. Impact on Large Data Sets

- If the StoreOrder table contains a large number of entries, the query performance will depend on the efficiency of indexing and how duplicates are handled. Using DISTINCT can incur some computational cost as the database ensures uniqueness in the result set.
- On the other hand, **the original join-based query might involve processing a larger intermediate result set**, which could slow down performance on large databases.

5. Recommendation:

- The simplified query is generally preferred if you're solely retrieving distinct User objects associated with a specific Item, as it directly queries the StoreOrder table and avoids unnecessary joins.
- To ensure optimal performance, make sure appropriate indexes exist on the StoreOrder table for the item_id and user_id columns.

The actual impact on performance depends on your database system (e.g., PostgreSQL, MySQL, Oracle) and its query optimization strategies. If needed, tools like query profiling or EXPLAIN plans can help analyze and confirm the efficiency of both approaches.

in the last version of the UserDAO, can you add a method that selects the collection of String encoding the username of any User associated with any StoreOrder for an Item identified by a parameter :item?

```
import javax.transaction.Transactional;
import java.util.List;

@Transactional
public class UserDAO extends BaseDAO {

    public void save(User user) {
        entityManager.persist(user);
    }

    public User findById(Long id) {
        return entityManager.find(User.class, id);
    }

    public List<User> findAll() {
        return entityManager.createQuery("SELECT u FROM User u",
User.class).getResultList();
    }

    public List<User> findUsersByItem(Item item) {
        return entityManager.createQuery(
            "SELECT DISTINCT s.user FROM StoreOrder s WHERE s.item =
:item", User.class)
            .setParameter("item", item)
            .getResultList();
    }

    // New method to retrieve the usernames
    public List<String> findUsernamesByItem(Item item) {
        return entityManager.createQuery(
            "SELECT DISTINCT s.user.username FROM StoreOrder s WHERE
s.item = :item", String.class)
            .setParameter("item", item)
            .getResultList();
    }

    public void delete(User user) {
        user = entityManager.merge(user);
        entityManager.remove(user);
    }
}
```

Key Changes:

1. **New Method**:
    - Added the findUsernamesByItem(Item item) method.
    - The JPQL query:

```
SELECT DISTINCT s.user.username FROM StoreOrder s WHERE s.item = :item
```

retrieves the username field from the User entity (s.user.username) associated with any StoreOrder that references the specified Item (s.item = :item).

    - The DISTINCT keyword ensures that duplicate usernames are excluded.
2. **Return Type**:
    - The method returns a List<String> containing only the username values.
3. **Efficiency**:
    - The query directly selects usernames (String values) instead of fetching entire User entities, reducing data overhead and improving query performance when only usernames are needed.

how could this be written using JOIN in the FROM clause?

```
SELECT DISTINCT u.username
FROM User u
JOIN StoreOrder s ON u.id = s.user.id
WHERE s.item = :item
```

Explanation:

1. **Explicit Join**:
    - The JOIN StoreOrder s ON u.id = s.user.id explicitly connects the User table (u) and the StoreOrder table (s) based on the foreign key (user.id) relationship.
2. **Selection**:
    - The SELECT DISTINCT u.username ensures that only distinct usernames are retrieved from the User entity.
3. **Condition**:
    - The WHERE s.item = :item specifies the condition that filters StoreOrder records based on the passed Item parameter (:item).

This explicit approach provides clarity in scenarios where you want full control over the joining process or when working with complex queries that might involve multiple tables and relationships.

Is there a difference in the queries that will be executed on the database?

Yes, there could be subtle differences in how the database executes the two queries, depending on the query optimizer and database engine being used.

Comparing the Queries

1. **Implicit Join Query**:

```
SELECT DISTINCT s.user.username
FROM StoreOrder s
WHERE s.item = :item
```

   - In this query, the relationship between StoreOrder and User is inferred using the JPA entity mapping (@ManyToOne).
   - **The database does not require an explicit JOIN clause because the association is already defined at the ORM level**. The query optimizer will still internally understand that s.user requires accessing the User table through the foreign key relationship (user_id).

2. **Explicit Join Query**:

```
SELECT DISTINCT u.username
FROM User u
JOIN StoreOrder s ON u.id = s.user.id
WHERE s.item = :item
```

   - Here, the JOIN is explicitly specified in the query, instructing the database to combine rows from the User and StoreOrder tables based on the foreign key condition (u.id = s.user.id).
   - The database has to perform an actual join operation explicitly as described in the query.

Possible Differences in Execution

1. Query Plan Generation

- **Implicit Join**: Since the join is implied by the JPA relationship, **the query optimizer decides how to handle it**, and it **may avoid redundant operations if the ORM** has already established some constraints.
- **Explicit Join**: The optimizer processes the explicit join, which may result in the same query execution plan. However, **in more complex queries, explicit joins give more control and make the logic clearer to both developers and the optimizer**.

2. Optimization and Performance

- In most modern databases, **the optimizer is smart enough to generate an equivalent execution plan for both queries**. The performance difference (if any) is typically negligible for straightforward queries like this one.
- In edge cases, explicit joins might perform slightly better if the database engine struggles to interpret implicit join conditions efficiently.

2 3. Readability and Complexity

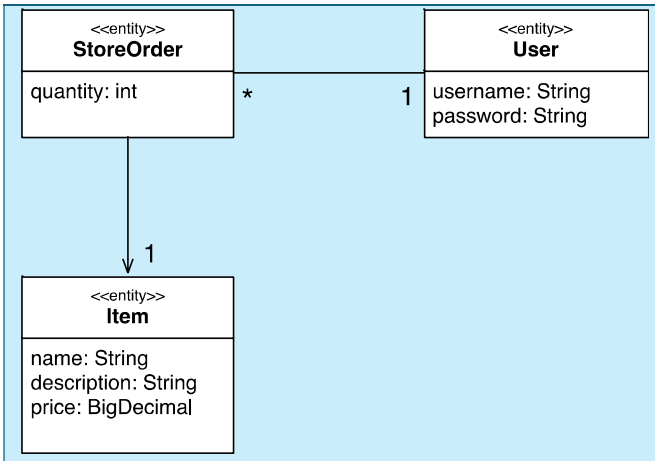
- While both queries achieve the same result, the explicit join is clearer in terms of specifying relationships, which can help developers and DBAs understand the query structure more easily. This becomes crucial for debugging or extending complex queries.

3 Practical Conclusion

In practice, there is **unlikely to be a significant performance difference** for this specific example in most database systems (e.g., PostgreSQL, MySQL, Oracle). Both queries will typically produce comparable execution plans. However:

- If you're aiming for clarity and control, explicit joins are recommended.
- If you trust the JPA abstraction to handle relationships correctly and want a concise query, the implicit join may suffice.

4 First round – demanding refinement - obsoleted



can you write the java classes that implement the following schema, with JPA annotations that map them on the database?

```
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.ManyToOne;
import javax.persistence.OneToMany;
```

```
import javax.persistence.Column;
import java.math.BigDecimal;
import java.util.List;

@Entity
public class StoreOrder {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private int quantity;

    @ManyToOne
    private User user;

    @OneToMany
    private List<Item> items;
FixMe: qui ha deviato dallo schema, in modo poco sensato:
va bene che un StoreOrder includa piu' Item
ma poi lo stesso Item dovrebbe potere essere in piu' Orders.
Quindi: deve essere @ManyTo…, lo schema dice che sia @…ToOne,
ma potrebbe anche andare @…ToMany
Questo poi ha conseguenza su come viene sviluppato tutto il seguito della
conversazione

    // Getters and setters
    …
}

@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String username;

    @Column(nullable = false)
    private String password;

FixMe: anche qui ha deviato dallo schema, ha omesso che uno User ha
riferimento ai suoi StoreOrders (che in una vera pratica sarebbe una
ottima scelta, che usa StoreOrders nella forma di un Mapper … ma non è lo
schema che avevamo passato
Anche questo poi ha conseguenza su come viene sviluppato tutto il seguito
della conversazione
Per correggere ci vuole aggiungere una collection di StoreOrder e questo
offre anche l'occasione di avere una associazione bidirezionale su cui
esercitare MappedBy e vedere che differenza produce nella costruzione del
database
```

```
    // Getters and setters
    …
}

@Entity
public class Item {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;
    @Column(nullable = false)
    private String description;
    @Column(nullable = false)
    private BigDecimal price;

    // Getters and setters…
    …
}
```

can you write the Data Access Objects associated with these classes, using JPA? In so doing, in tha DAO of the class User, can you also include a method that selects all the Users that have at least one Order for a given Item?

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import java.util.List;

public interface UserDAO extends JpaRepository<User, Long> {

    @Query("SELECT DISTINCT u FROM User u JOIN u.storeOrders o JOIN
o.items i WHERE i.id = :itemId")
    List<User> findUsersWithOrdersForItem(@Param("itemId") Long itemId);
}
```

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface ItemDAO extends JpaRepository<Item, Long> {
    // Add any custom methods here if needed
}
```

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface StoreOrderDAO extends JpaRepository<StoreOrder, Long> {
    // Add any custom methods here if needed
```

```
}
```

Explanation:

1. UserDAO:
    - The method findUsersWithOrdersForItem is a custom query. It uses JPQL to join the User, StoreOrder, and Item entities.
    - The :itemId parameter filters users who have orders containing the given item.
2. ItemDAO and StoreOrderDAO:
    - These DAOs inherit basic CRUD operations from JpaRepository. You can extend them with additional custom queries if necessary.

can you write these classes using plain JPA (not Spring), and have queries expressed in JPQL?

```
import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
import javax.persistence.Query;
import java.util.List;

public class UserDAO {
    @PersistenceContext
    private EntityManager entityManager;

    public UserDAO(EntityManager entityManager) {
        this.entityManager = entityManager;
    }

    public void createUser(User user) {
        entityManager.getTransaction().begin();
        entityManager.persist(user);
        entityManager.getTransaction().commit();
    }

    public User findUserById(Long id) {
        return entityManager.find(User.class, id);
    }

    public List<User> findUsersWithOrdersForItem(Long itemId) {
        String jpql = "SELECT DISTINCT u FROM User u JOIN StoreOrder o ON
u.id = o.user.id JOIN o.items i WHERE i.id = :itemId";
        Query query = entityManager.createQuery(jpql);
        query.setParameter("itemId", itemId);
        return query.getResultList();
    }
}
```

5 Key Notes:

1. **EntityManager**:
   - Each DAO class uses EntityManager for JPA operations, allowing flexibility in standalone JPA applications without Spring.
2. **Transaction Management**:
   - Explicit transactions are handled using entityManager.getTransaction().
3. **Custom JPQL in** UserDAO:

- The query findUsersWithOrdersForItem selects users associated with orders that contain the specified item ID.

in the UserDAO, can you rewrite the following query without using native SQL and rather using only JPQL?

String jpql = "SELECT DISTINCT u FROM User u JOIN StoreOrder o ON u.id = o.user.id JOIN o.items i WHERE i.id = :itemId";

```
String jpql = "SELECT DISTINCT u FROM User u JOIN u.storeOrders o JOIN
o.items i WHERE i.id = :itemId";

FixMe: il codice di User non includeva una collezione di StoreOrders …
che invece lui ora assume (vedi sotto)
```

6 Explanation:

1. JPQL operates on entity relationships defined in your Java classes rather than using explicit join conditions like ON.
2. The rewritten query leverages the mapped relationships (storeOrders in User and items in StoreOrder) to navigate between entities directly.

This query assumes that:

- There is a List<StoreOrder> or similar collection field in the User entity.
- There is a List<Item> or similar collection field in the StoreOrder entity.

could also the following form implement the same selection?

SELECT s.buyer FROM StoreOrder s WHERE s.item = :item

The query you provided could indeed implement a selection for users who placed orders that contain a specified item. However, it depends on how the entities and relationships are mapped in your JPA model.

Here's the query you shared:

```
SELECT s.buyer
FROM StoreOrder s
WHERE s.item = :item
```

This form assumes:

1. The StoreOrder entity has a field named item mapped to the items it contains.

2. The buyer field in StoreOrder is directly linked to the User entity.

If your entity relationships match this structure, then this query is valid. However, in your original schema, it seems like StoreOrder has a relationship with a list of items (List<Item>), and User is linked to StoreOrder. Therefore, the JPQL query that matches your current model would look more like this:

```
String jpql = "SELECT DISTINCT u FROM User u JOIN u.storeOrders o JOIN
o.items i WHERE i = :item";
```

This query navigates through the mapped relationships (storeOrders in User and items in StoreOrder) to filter users based on the item.

What are the performance differences between these two queries?

To assess the performance differences between these two queries, let's consider their structures and how they interact with the database.

1. Query A:

java

```
String jpql = "SELECT DISTINCT u FROM User u JOIN u.storeOrders o JOIN
o.items i WHERE i.id = :itemId";
```

Performance considerations:

- **Join Navigation:** This query starts with the User entity and joins it with the related entities StoreOrder and Item. It will utilize the relationships and indexes (if properly defined in the database schema) to filter users efficiently.
- **Result Set Reduction:** The DISTINCT keyword ensures that duplicate users are eliminated after the filtering process, which adds some overhead.
- **Advantage:** If the number of User entities is relatively small compared to the size of the Item and StoreOrder entities, this query may perform faster as it starts by navigating through the User table and applies constraints later.

2. Query B:

java

```
String jpql = "SELECT s.buyer FROM StoreOrder s WHERE :item MEMBER OF
s.items";
```

Performance considerations:

- **Collection Predicate:** The MEMBER OF clause checks whether the given item belongs to the items collection in the StoreOrder entity. This can lead to additional processing overhead, especially if the items collection is large and unindexed.
- **Direct Focus on Orders:** This query starts with StoreOrder, directly filtering by the given item and then retrieving the associated buyer (i.e., User). If there are fewer StoreOrder entries compared to users or items, this query may perform better as it focuses on filtering a smaller subset of data.

- **No DISTINCT Needed:** If the buyer field is one-to-one with StoreOrder, there's no need for DISTINCT, which avoids that overhead.

General Comparison:

- **Query A** might perform better in scenarios where:
    - The User entity has fewer rows than the StoreOrder or Item entities.
    - The relationships (storeOrders and items) are well-indexed and navigable.
- **Query B** might perform better in scenarios where:
    - There are far fewer StoreOrder rows than User rows.
    - The items collection is not overly large or is well-optimized with database indexes.

Real-World Impact:

The actual performance difference depends on factors such as:

- The size of the dataset for each entity (User, StoreOrder, Item).
- Indexing strategies in the database (e.g., whether itemId or collection fields are indexed).
- Query optimization by the JPA implementation and the underlying database engine.

To decide which query is faster in your specific use case, you would likely need to:

1. Analyze the database schema and relationships.
2. Use a profiling tool to measure execution times for both queries against your data.
3. Check the database query plans for each query to see how indexes and joins are utilized.

7 Further experiments

Poi sperimentare con cascading e greedy/lazy load e vedere come un unico find() recupera tutta la parte utile ... e chiedere quali sono le query implementate sul db

e.g. cascading su remove

can you write the java classes that implement the following schema, with JPA annotations that map them on the database?